

CSC111 Project Proposal: TODO FILL IN YOUR PROJECT TITLE HERE

TODO: FILL IN ALL GROUP MEMBER NAMES HERE

March 2, 2026

Problem Description and Research Question

TODO

Computational Plan

1. Parse `courses.html` into `courses.json`, where each course entry stores its course code, raw prerequisite string, and raw exclusion list.
2. For each course, parse the raw prerequisite string into a structured, evaluable form:
 - a) Extract course codes via regex `([A-Z]{3}[0-9]{3}[HY][0-9])`.
 - b) Infer logical structure from UofT's formatting conventions: `,` as AND, `/` as OR, parentheses as grouping.
 - c) Classify non-course requirements (credit thresholds, permissions, placement tests) by keyword detection.
3. Implement a custom `CourseTree` class where each node stores a student state (completed courses, total credits). The tree models course selection analogously to `a2_minichess.py`: the current state determines available moves (eligible courses), and selecting a course expands the tree to a new child state.
4. Implement core tree methods:
 - a) `get_available_courses(state)` — return all available courses at a current state based on prerequisites/exclusions.
 - b) `append(course)` — TODO
 - c) `find_path_to(target)` — search the tree for a sequence of courses that makes the target course eligible, analogous to searching a game tree for a winning position.
 - d) `get_credits(optional[Dept])` — return the number of credits satisfied by the current tree, taking an optional argument to check by department for some requirements.
5. Build an interactive user interface where a student inputs completed courses, views available courses (children of the current node), selects courses to “take” (expanding the tree), and queries for paths to a target course.

References